PROGRAMACIÓN CONCURRENTE Y DISTRIBUIDA

GUIÓN 2: SEMÁFOROS EN JAVA

1. Exclusión mutua usando espera ocupada

En el siguiente ejemplo tratamos de proteger una sección crítica usando una variable compartida `v` que indica el estado de la sección crítica (0 indica libre y 1 ocupada). La variable está inicializada a 0 ya que inicialmente la sección crítica está libre. El proceso que entra en la sección crítica pone la variable a 1 de forma que cualquier otro proceso no podrá entrar ya que quedará detenido en la sentencia `while`. Cuando el proceso que está en la sección crítica sale, pone de nuevo la variable `v` a 0 indicando que deja libre la sección crítica.

```
/* fichero PrimerIntento.java */

public class PrimerIntento {

    public static volatile int v = 0;

    /**
     * @param args
     */
    public static void main(String[] args) {

        Thread a = new Hilo();
        Thread b = new Hilo();

        a.start();
        b.start();

        try {
            a.join();
            b.join();
        } catch (InterruptedException e) {
            e.printStackTrace();
        }

        System.out.println("Finalizando hilo principal");
    }
}

class Hilo extends Thread {

    public void run() {

        for (int i = 0; i < 10; i++) {

            while (PrimerIntento.v == 1) {
                ; // espera activa
            }

            try {
                Thread.sleep(Math.round((Math.random() * 1000)));
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
```

```
            PrimerIntento.v = 1;

            System.out.println(
                "Entrando    a    la    seccion    critica    "    +
Thread.currentThread().getName()
            );

            try {
                Thread.sleep(Math.round((Math.random() * 1000)));
            } catch (InterruptedException e) {
                e.printStackTrace();
            }

            System.out.println(
                "Saliendo    de    la    seccion    critica    "    +
Thread.currentThread().getName()
            );

            PrimerIntento.v = 0;
        }
    }
}
```

En el ejemplo anterior se muestra la única manera de compartir variables de tipos simples entre hilos que es mediante la declaración de una variable estática. Si quisiéramos compartir objetos, también se podrían pasar como parámetro a los diferentes hilos.

Ejercicio 1. Ejecuta el programa anterior. ¿Se obtiene el resultado esperado? Justifica la respuesta.

2. Exclusión mutua usando synchronized.

Por defecto, la ejecución del código de los métodos de un objeto no está protegida y cualquier número de hilos puede estar ejecutando simultáneamente el mismo, o distintos métodos, del mismo objeto. El código de los métodos ejecutados simultáneamente puede acceder a las mismas variables, por lo tanto, dicho código puede considerarse una sección crítica. Java proporciona una forma de conseguir que un método, o un bloque de código se ejecute en exclusión mutua y es mediante el uso de `synchronized`.

Synchronized aplicado a un método.

Un método puede llevar el modificador `synchronized`. Todos los métodos con este modificador se ejecutarán en exclusión mutua entre sí. Cuando un método sincronizado se está ejecutando sobre un objeto concreto, ningún otro método sincronizado podrá ejecutarse sobre ese mismo objeto. Sin embargo, cualquier otro método no sincronizado sí podrá ejecutarse. Por ejemplo, supongamos la siguiente clase:

```
public class Sincronizados {

    public void m1() {
    }

    public synchronized void m2() {
    }
```

```
    public void m3() {
    }

    public synchronized void m4() {
    }

    public void m5() {
    }
}
```

En el ejemplo, sólo puede haber un hilo ejecutando el método 2 o el 4, mientras que se pueden estar ejecutando cualquier número de hilos en el resto de métodos. Cualquier hilo que quiera ejecutar un método sincronizado mientras otro lo está ejecutando tendrá que esperar a que éste termine.

Es como si el objeto tuviera un cerrojo (semáforo binario). Cuando un hilo está ejecutando un método sincronizado, el cerrojo asociado a ese objeto está cerrado y ningún otro hilo puede ejecutar otro método sincronizado. Cuando el primer método acaba se libera el cerrojo, entonces algún hilo que estaba esperando dicha liberación podrá ejecutar el método sincronizado correspondiente.

Veamos el uso de `synchronized` en un ejemplo donde dos hilos comparten una variable y la incrementan en 1000 unidades cada uno.

```
class VariableCompartida {

    private int var;

    public VariableCompartida(int val) {
        var = val;
    }

    public int getVar() {
        return var;
    }

    public synchronized void incrementa() {
        int temp;

        temp = var;

        try {
            Thread.sleep((int) Math.round(Math.random()));
        } catch (InterruptedException e) {
            e.printStackTrace();
        }

        temp++;
        var = temp;
    }
}

class Hilo extends Thread {
```

```
    private String id;
    private VariableCompartida compar;

    public Hilo(String _id, VariableCompartida a) {
        id = _id;
        compar = a;
    }

    public void run() {
        for (int i = 0; i < 1000; i++) {
            compar.incrementa();
            System.out.println(id + " " + compar.getVar());
        }
    }
}

public class Ejemplo {

    /**
     * @param args
     */
    public static void main(String[] args) {

        VariableCompartida compar = new VariableCompartida(0);

        Hilo a = new Hilo("Hilo a", compar);
        Hilo b = new Hilo("Hilo b", compar);

        a.start();
        b.start();

        try {
            a.join();
            b.join();
        } catch (InterruptedException e) {
            e.printStackTrace();
        }

        System.out.println("Hilo principal " + compar.getVar());
    }
}
```

Ejercicio 2. Ejecuta el programa anterior eliminando el modificador synchronized del método incrementar. Ejecútalo de nuevo insertando el modificador. Justificar los resultados obtenidos.

Synchronized aplicado a un bloque de código.

A veces no nos interesa que todo el método sea sincronizado, sino solamente una parte de él. En ese caso, `synchronized` se puede usar sobre un bloque de código. Para ello es necesario indicar el objeto que será utilizado como cerrojo. Esto hace que el bloque de

código se ejecute en exclusión mutua con cualquier bloque de código o método sincronizado sobre el mismo objeto.

Ya que `synchronized` se puede aplicar sobre un bloque de código indicando un objeto, podríamos obtener el equivalente a una sección crítica protegida por un semáforo binario, creando un objeto vacío para usarlo sólo como cerrojo. Cualquier bloque sincronizado sobre este objeto en cualquier hilo será ejecutado en exclusión mutua. Lo importante es que dicho objeto sea una variable compartida por todos los hilos cuyas secciones críticas vayamos a proteger con bloques `synchronized`.

```
Object obj=new Object();

synchronized(obj){
    // sección crítica
}
```

Ejercicio 3: Resolver el ejercicio anterior usando un bloque `synchronized`.

3. Exclusión mutua usando la clase ReentrantLock (Cerrojo).

En el paquete `java.util.concurrent.locks` podemos encontrar tanto la clase `ReentrantLock` como la interfaz que ésta implementa y que se llama `Lock`. Las implementaciones de `Lock` proporcionan más funcionalidad que la obtenida con métodos y bloques `synchronized`. Un cerrojo permite controlar el acceso en exclusión mutua a recursos compartidos por múltiples hilos: sólo un hilo puede adquirir el cerrojo y para poder acceder al recurso compartido un hilo debe adquirir en primer lugar dicho cerrojo. Podemos ver un objeto de tipo `ReentrantLock` como un semáforo binario.
Los métodos y bloques `synchronized` proporcionan implícitamente un cerrojo asociado con cada objeto, pero la forma de adquirirlo y liberarlo es de una forma estructurada en bloques: si debemos adquirir varios cerrojos, estos deben ser liberados en el orden opuesto y dentro del ámbito en el que fueron adquiridos. Aunque esta forma de programar evita muchos errores típicos de programación usando cerrojos, a veces es conveniente poder trabajar con cerrojos de una forma más flexible. Por ejemplo, adquirimos el cerrojo del objeto A, después del objeto B, liberamos A, adquirimos C, liberamos B, … Las implementaciones de la interfaz `Lock` permiten que las operaciones de adquirir y liberar un cerrojo estén en diferentes ámbitos y permiten que varios cerrojos sean adquiridos y liberados en cualquier orden.
La estructura común de trabajo con implementaciones de `Lock` es:

```
Lock l = ...;

l.lock();
try {
    // acceso al recurso protegido
} finally {
    l.unlock();
}
```

En esta práctica vamos a trabajar con la Clase `ReentrantLock` que implementa la interfaz `Lock`. De esta forma, la estructura anterior quedaría como:

```
Lock l = new ReentrantLock();

l.lock();
try {
    // acceso al recurso protegido
} finally {
    l.unlock();
}
```

Veamos de nuevo el problema de la variable compartida por dos hilos usando la clase `ReentrantLock`:

```
import java.util.concurrent.locks.ReentrantLock;

class VariableCompartida {

    private int var;
    private ReentrantLock l = new ReentrantLock();

    public VariableCompartida(int val) {
        var = val;
    }

    public int getVar() {
        return var;
    }

    public void incrementar() {
        int temp;

        l.lock();
        try {
            temp = var;

            try {
                Thread.sleep((int) Math.round(Math.random()));
            } catch (InterruptedException e) {
                e.printStackTrace();
            }

            temp++;
            var = temp;

        } finally {
            l.unlock();
        }
    }
}

class Hilo extends Thread {

    private String id;
    VariableCompartida compar;

    public Hilo(String _id, VariableCompartida a) {
        id = _id;
        compar = a;
```

```
    }

    public void run() {
        for (int i = 0; i < 1000; i++) {
            compar.incrementar();
            System.out.println(id + " " + compar.getVar());
        }
    }
}
```

```
public class Ejemplo {

    /**
     * @param args
     */
    public static void main(String[] args) {

        VariableCompartida compar = new VariableCompartida(0);

        Hilo a = new Hilo("Hilo a", compar);
        Hilo b = new Hilo("Hilo b", compar);

        a.start();
        b.start();

        try {
            a.join();
            b.join();
        } catch (InterruptedException e) {
            e.printStackTrace();
        }

        System.out.println("Hilo principal " + compar.getVar());
    }
}
```

4. Exclusión mutua y sincronización usando la clase Semaphore

La clase `Semaphore` está definida en el paquete `java.util.concurrent`. Cuando creamos un objeto de la clase `Semaphore` indicamos su valor inicial. Las operaciones wait y signal tienen su equivalente en los métodos denominados `acquire` y `release` respectivamente. Una variable de tipo `Semaphore` representa un semáforo general. Veamos cómo resolver el problema de la variable compartida.

```
import java.util.concurrent.Semaphore;

class Hilo extends Thread {

    private String id;

    public Hilo(String _id) {
        id = _id;
    }
```

```
    public void run() {

        int temp;

        for (int i = 0; i < 1000; i++) {
            try {
                PruebaSemaforo.mutex.acquire();
            } catch (InterruptedException e) {
                // manejar la excepción si se desea
            }

            temp = PruebaSemaforo.n;
            temp++;
            PruebaSemaforo.n = temp;

            PruebaSemaforo.mutex.release();
        }

        System.out.println(id + " " + PruebaSemaforo.n);
    }
}

public class PruebaSemaforo {

    public static int n = 0;
    public static Semaphore mutex = new Semaphore(1);

    /**
     * @param args
     */
    public static void main(String[] args) {

        Thread a = new Hilo("Hilo a");
        Thread b = new Hilo("Hilo b");

        a.start();
        b.start();

        try {
            a.join();
            b.join();
        } catch (InterruptedException e) {
            e.printStackTrace();
        }

        System.out.println("Fin del hilo principal " + n);
    }
}
```

Es importante recordar que estos semáforos son generales, por lo tanto, para conseguir exclusión mutua con ellos debemos asegurarnos de hacer los wait y signal (`acquire` y `release`) en parejas. A diferencia de los cerrojos tipo `Lock`, en estos sí se acumulan los signal incrementando el valor del semáforo.

Veamos ahora un ejemplo de sincronización usando semáforos generales implementados con `Semaphore`.

```
import java.util.concurrent.Semaphore;

class HiloHola extends Thread {

    public void run() {
        for (int i = 0; i < 10; i++) {
            try {
                HolaMundo.hola.acquire();
            } catch (InterruptedException e) {
                // manejar la excepción si se desea
            }

            System.out.print("Hola ");
            HolaMundo.mundo.release();
        }
    }
}

class HiloMundo extends Thread {

    public void run() {
        for (int i = 0; i < 10; i++) {
            try {
                HolaMundo.mundo.acquire();
            } catch (InterruptedException e) {
                // manejar la excepción si se desea
            }

            System.out.println("mundo");
            HolaMundo.hola.release();
        }
    }
}

public class HolaMundo {

    public static Semaphore hola = new Semaphore(1);
    public static Semaphore mundo = new Semaphore(0);

    /**
     * @param args
     */
    public static void main(String[] args) {

        Thread a = new HiloHola();
        Thread b = new HiloMundo();

        a.start();
        b.start();

        try {
            a.join();
            b.join();
```

```
        } catch (InterruptedException e) {
            e.printStackTrace();
        }

        System.out.println("Fin del hilo principal ");
    }
}
```

Fíjate lo importante que resulta inicializar bien los semáforos. En este ejemplo se consigue que el hilo `b` no muestre la palabra mundo hasta que no se ha mostrado la palabra hola gracias a que el semáforo `mundo` está inicializado a 0.

Ejercicio 4: Usando semáforos, escribe un programa que imprima correctamente en pantalla la frase "hola amigos del mundo" 10 veces y cada frase en una línea. Cada palabra de la frase debe imprimirla un hilo distinto.

Ejercicio 5: Usando la clase Semaphore escribe un programa que resuelva el problema clásico del Productor/Consumidor con buffer limitado.